

OOPJ Day 8

Wed, 16 Sept 26

Switch Expressions vs. Return Statements

- yield inside a switch expression returns a value for the switch block, stored in a local variable
- return inside a switch exits the enclosing method entirely, not just the switch
- Syntactically: `result = switch(...) { case X -> { yield value; } }` stores the yielded value
- Using a plain return statement inside the same block returns from the function, bypassing the switch's value assignment
- Key rule: yield is scoped to the switch; return is scoped to the method

Sealed, Non-Sealed, and Final Classes (Java 16/17)

- Three classic inheritance modes before Java 16:
 - Regular class: any class can extend it freely
 - Abstract class: cannot be instantiated; forces subclassing
 - Final class: cannot be extended by anyone
- Java 16/17 introduced **sealed classes** for conditional, restricted inheritance
 - Syntax: sealed class Employee permits Manager, Engineer
 - Only the explicitly listed classes may extend Employee
 - Any unlisted class attempting to extend a sealed parent causes a compile error
- Every direct subclass of a sealed class must declare exactly one of three qualifiers:
 - final: that subclass cannot be extended further
 - sealed: that subclass is itself sealed and must provide its own permits list
 - non-sealed: reopens inheritance freely; any class may extend it
- Practical example:
 - Employee is sealed, permits Manager and Engineer
 - Engineer declared non-sealed → ComputerEngineer, MechanicalEngineer, etc. can all extend Engineer freely
 - Engineer declared final → no further subclassing allowed
 - Engineer declared sealed permits ComputerEngineer → only ComputerEngineer may extend Engineer
- C++ analogy: no direct equivalent; closest concept is controlling visibility of constructors

Strings, String Pool, and Immutability

- Every double-quoted value in Java is automatically an instance of the String class
- Java uses the Unicode character set (2 bytes per character) vs. ASCII in C/C++ (1 byte)
 - This doubles memory consumption for strings compared to C/C++

- To manage this, Java maintains a **String Pool** (a dedicated block on the heap)
 - The pool stores one canonical copy of each unique string value
 - A lookup table maps string content → memory address
 - When a string literal is created, Java checks the pool first; if found, the existing object is reused
 - A new object is only created if the content is different, or the programmer explicitly uses `new String("hello")`
- Strings are **immutable** (non-changeable, constant) because pool objects are shared
 - If two references point to the same pooled object and one could modify it, the other reference's data would be silently corrupted
 - This is the same shallow copy vs. deep copy concern discussed in the context of object references
- `==` on strings compares **memory addresses** (references), not content
 - Two literals with the same value share one pool object → `==` returns true
 - Two objects created with `new String("hello")` sit at different addresses → `==` returns false
- `.equals()` compares the **content** of two string objects regardless of address
- `.equalsIgnoreCase()` compares content while ignoring uppercase/lowercase differences
- Java does not support operator overloading (unlike C++), so `==` always means reference comparison, never content comparison

StringBuffer, toString(), and Efficient Concatenation

- String concatenation with `+` creates multiple intermediate String objects
 - Example: `"hello" + "bye"` creates one object for "hello", one for "bye", one for "helloBye" = 3 objects total
 - In a complex display method with 7–8 fields concatenated, 7–8+ objects are created unnecessarily
- **StringBuffer** is a mutable string class designed to avoid this overhead
 - Declared as: `StringBuffer buffer = new StringBuffer(initialString);`
 - Fields appended with: `buffer.append(value)` — accepts any type (String, int, object, etc.)
 - Final string retrieved with: `buffer.toString()` which converts the buffer to a String
- Practical use: override `toString()` in custom classes using StringBuffer for efficient output
- **toString() in the Object class:**
 - Default behavior: returns `ClassName@hashCode` (e.g., `Data@1a2b3c`)
 - When any object is passed to `System.out.println()`, the compiler internally calls `.toString()` on it
 - If the class does not override `toString()`, the call travels up to Object's default implementation
 - Override `toString()` in your class to return meaningful field values instead
 - Example override returns: `"number1=5, number2=10, status=active"`
- `super.toString()` calls the parent class's `toString()` implementation

- Useful in inheritance chains: Manager can call `super.toString()` to include Employee fields, then append its own

The Object Class and Core Methods

- Every Java class implicitly extends `java.lang.Object` — the compiler inserts `extends java.lang.Object` automatically
- Object is the root of the entire Java class hierarchy
- Key methods provided by Object (all overridable unless declared `final`):
 - `getClass()`: returns runtime class information (used for RTTI — Run-Time Type Information)
 - `hashCode()`: returns a unique integer generated from the object's content
 - Two objects with identical data produce the same hash code
 - Used extensively in searching and sorting algorithms
 - More efficient than member-wise comparison for complex objects with many fields
 - Example: instead of comparing 10–15 fields individually, compare two hash codes
 - `equals()`: by default compares references (memory addresses), not content
 - Override to implement content-based comparison
 - String's `equals()` uses hash codes to compare content
 - `clone()`: creates a duplicate copy of the object (equivalent to deep copy / copy constructor in C++)
 - `toString()`: returns string representation; default is `ClassName@hashCode`; override for custom output
 - `wait()`, `notify()`, `notifyAll()`: used for thread synchronization in multithreaded applications; covered later
 - `finalize()`: deprecated since Java 6/7, fully removed in Java 9 (see Garbage Collection section)
- **Deprecated** means the Java language recommends not using the method due to bugs or issues
 - The method still exists for backward compatibility but should be avoided
 - Eclipse shows a strikethrough on deprecated methods

Packages, Modules, and Access Specifiers

- **Class**: groups primitive types, object types, and methods that act on them
- **Package**: groups classes, interfaces, enums, and annotations together
 - Syntax: `package org.pankaj;`
- **Module** (Java 9+): groups a series of packages
 - Example: `java.base` is a module containing many core packages
 - C++ analogy: roughly comparable to a namespace, but a module is a collection of namespaces
- Two access specifiers available at the class level:

- public: class is accessible from any package
 - Default (no specifier): class is only accessible within its own package
- Importing classes from other packages:
 - Fully qualified name: `org.pankaj.util.DateCalculator obj = new org.pankaj.util.DateCalculator();`
 - Import statement: `import org.pankaj.util.DateCalculator;`
 - Wildcard: `import org.pankaj.*;` — imports all public classes in that package only
 - Does **not** import classes from child packages (e.g., `org.pankaj.util` is not included)
 - To include child packages, each must be imported separately
 - Prefer individual imports over wildcards for readability
 - Eclipse shortcut: `Ctrl+Shift+O` auto-organizes imports
- `java.lang` is automatically imported in every Java file
 - Contains: `String`, `System`, `Integer`, and all other wrapper classes, among others

Pattern Matching, Wrapper Classes, and Boxing/Unboxing

- **instanceof before Java 16:**
 - Check type: `if (arr[i] instanceof Manager)`
 - Then manually downcast: `Manager objManager = (Manager) arr[i];`
 - Two separate statements required
- **Pattern matching (Java 16+):**
 - Combines type check and downcast: `if (arr[i] instanceof Manager objManager)`
 - If the condition is true, `objManager` is automatically the downcasted reference
 - No manual cast needed; cleaner and less error-prone
 - C++ equivalent: `dynamic_cast<Manager*>(employee)` — same concept, different syntax
- **Java is not 100% object-oriented** because it supports primitive data types (`int`, `float`, `char`, etc.)
 - Primitives are not objects and cannot use dot notation or be passed as `Object` references
 - Solution: **wrapper classes** (`Integer`, `Double`, `Float`, `Character`, etc.)
- **Boxing**: converting a primitive into its wrapper object
 - Old syntax: `Integer data = (Integer) value;` — explicit cast
 - Auto-boxing (modern Java): `Integer data = value;` — compiler handles it automatically
- **Unboxing**: converting a wrapper object back to a primitive
 - Old syntax: `int num = (int) data;`
 - Auto-unboxing: `int num = data;` — compiler handles it automatically
- Boxed values are **immutable**, stored in a **constant pool** (analogous to the `String` pool)
 - Two final variables with the same value point to the same pool entry
 - Changing one does not affect the other; they are independent copies
- Practical use of wrapper classes:
 - Write a generic `display(Object obj)` method that accepts any Java object

- Since Integer → Number → Object and Employee → Object, any type can be passed
- Use instanceof / pattern matching inside to handle specific types
- **Singleton Design Pattern** (briefly introduced):
 - Ensures only one instance of a class exists at any time
 - Implementation: make the constructor private + provide a static factory method that returns the single instance
 - The static method tracks whether an object already exists; if so, returns it; if not, creates one

Memory Management, Garbage Collection, and Finalization

- **C/C++ memory problems** (motivation for Java's approach):
 - **Dangling pointer**: a pointer whose referenced memory has been freed or gone out of scope
 - The pointer still holds the old address, but that memory is no longer valid
 - Dereferencing it yields garbage or crashes
 - **Memory leak**: dynamically allocated memory with no pointer referencing it
 - No one knows the address → no one can free it → memory is wasted until program terminates
 - Critical for long-running applications (Google, Amazon, IRCTC) that run 24/7
 - Accumulated leaks eventually exhaust system memory, requiring a restart (downtime)
 - Root cause: programmer is responsible for both allocating (new/malloc) and releasing (delete/free) memory
 - Release too early → dangling pointer
 - Never release → memory leak
- **Java's solution**:
 - All objects must be allocated on the **managed heap** using the new operator
 - Java provides **no delete operator** → eliminates dangling pointer risk
 - No delete → no destructors (unlike C++)
 - The JVM manages the heap and tracks all object references
- **Reference Counting**:
 - JVM maintains a **Reference Count (RC) table**: maps each heap address to a count of references pointing to it
 - Count increments when a new reference points to the object
 - Count decrements when a reference is set to null, reassigned to another object, or goes out of scope
 - When count reaches zero → object is marked for garbage collection
 - Analogy: three flatmates share a library book; each writes a note when they finish using it; the last one to finish returns it
- **Garbage Collector (GC)**:
 - A low-priority background thread (part of the JVM) that runs when the JVM is idle

- Identifies all objects whose reference count has fallen to zero
- Destroys those objects and reclaims their memory
- Programmer cannot directly invoke the GC
- `System.gc()` sends a request to increase GC thread priority — no guarantee it will run
- Calling `System.gc()` 100 times makes no difference if the JVM is busy
- **Memory Pool:**
 - At JVM startup, a chunk of memory is pre-allocated from the OS (the memory pool)
 - All new object allocations draw from this pool
 - When the pool runs low, JVM first invokes GC to reclaim memory; if still insufficient, requests more from the OS
 - Avoids constant OS requests for every individual allocation (improves performance)
- **finalize() method:**
 - Defined in `Object` class; intended to be called by the GC just before an object is destroyed
 - Equivalent role to a destructor in C++
 - Deprecated in Java 6/7; fully removed in Java 9
 - Problem: no guarantee when or even if `finalize()` will be called
 - If the program terminates before GC runs, `finalize()` may never fire
 - Cannot be relied upon for resource release (file handles, DB connections, etc.)
 - **Finalization process:** JVM calls `finalize()` on all zero-RC objects just before GC destroys them — but this is best-effort only
 - Best practice: define your own explicit resource-release method and call it deterministically
- **Notes to write down (as dictated):**
 - In C/C++, dynamically allocated memory must be explicitly released by the programmer using `free` © or `delete` (C++)
 - Releasing too early causes a dangling pointer; never releasing causes a memory leak
 - Java eliminates the `delete` operator and manages object destruction via the garbage collector
 - GC uses reference counting; when count falls to zero, the object is marked for collection
 - GC is a low-priority background thread; `System.gc()` requests higher priority but offers no guarantee
 - All Java objects are allocated on the managed heap and destroyed when reference count falls to zero
 - Java does not support destructors because destruction timing is non-deterministic
 - `Object` class provides `finalize()`; ideally called just before destruction, but unreliable; do not rely on it

Schedule and Upcoming Topics

- Tomorrow (17th Sep): exception handling (full session)
- 18th Sep: session may or may not be conducted; Pankaj to confirm with Anika
 - If cancelled, compensated on Sunday
- From 19th Sep onwards: session timings change to **4:30–8:30 PM** (evening)
 - Morning slot no longer available due to Pankaj's offline batch commitment
 - This change was pre-communicated to Anika and the center
- Remaining topics to cover:
 - Exception handling
 - File operations
 - Generics
 - Collections framework (usage-focused; internals covered in DSA module)
 - Functional programming and lambda expressions
 - Multithreading
 - JDBC (required for web technologies)
 - Records, interfaces, static imports (Java 16 features — briefly deferred)
- Progress: ~32 hours completed, ~30 hours remaining (~50% of module done)
- Exercises may be shared before tomorrow's session; check the group

Chat with meeting transcript:

<https://notes.granola.ai/t/bec55d8c-dbf8-40f8-b0c6-a65a3570f0f9-00demib2>